

Content security policy

Content Security Policy can significantly reduce the risk and impact of cross-site scripting attacks in modern browsers.



Joe Medley



(<https://github.com/jpmedley>)

Mike West



(<https://twitter.com/mikewest>)



(<https://mikewest.org/>)

Browser Support



25



14



23



7

Source (<https://developer.mozilla.org/docs/Web/HTTP/Reference/Headers/Content-Security-Policy>)

The web's security model is based on a *same-origin policy*

(https://en.wikipedia.org/wiki/Same-origin_policy). For example, code from `https://mybank.com` must have access to only `https://mybank.com`'s data, and `https://evil.example.com` must never be allowed access. Each origin is, in theory, kept isolated from the rest of the web, giving developers a safe sandbox to build in. In practice, however, attackers have found several ways to subvert the system.

Cross-site scripting (XSS) (https://en.wikipedia.org/wiki/Cross-site_scripting) attacks, for example, bypass the same-origin policy by tricking a site into delivering malicious code along with the intended content. This is a huge problem, as browsers trust all of the code that shows up on a page as being legitimately part of that page's security origin. The XSS Cheat Sheet (https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet) is an old but representative cross-section of the methods an attacker might use to violate this trust by injecting malicious code. If an attacker successfully injects *any* code at all, they've compromised the user session and gained access to private information.

This page outlines Content Security Policy (CSP) as a strategy for reducing the risk and impact of XSS attacks in modern browsers.

Components of CSP

To implement an effective CSP, take the following steps:

- Use allowlists to tell the client what's allowed and what isn't.
- Learn what directives are available.
- Learn the keywords they take.
- Restrict the use of inline code and `eval()`.
- Report policy violations to your server before enforcing them.

Source allowlists

XSS attacks exploit the browser's inability to distinguish between script that's part of your application and script that's been maliciously injected by a third party. For example, the Google +1 button at the bottom of this page loads and executes code from `https://apis.google.com/js/plusone.js` in the context of this page's origin. We trust that code, but we can't expect the browser to figure out on its own that code from `apis.google.com` is safe to run, while code from `apis.evil.example.com` probably isn't. The browser happily downloads and executes any code a page requests, regardless of source.

CSP's `Content-Security-Policy` HTTP header lets you create an allowlist of sources of trusted content, and tells the browser to execute or render only resources from those sources. Even if an attacker can find a hole to inject a script through, the script won't match the allowlist, and therefore won't be executed.

We trust `apis.google.com` to deliver valid code, and we trust ourselves to do the same. Here's an example policy that allows scripts to execute only when they come from one of those two sources:

Content-Security-Policy: script-src 'self' https://apis.google.com

`script-src` is a directive that controls a set of script-related privileges for a page. This header 'self' as one valid source of script, and `https://apis.google.com` as another. The browser can now download and execute JavaScript from `apis.google.com` over HTTPS, as well as from the current page's origin, but not from any other origin. If an attacker injects code into your site, the browser throws an error and doesn't run the injected script.



The console shows an error when a script tries to run from an origin not on the allowlist.

Policy applies to a wide variety of resources

CSP provides a set of policy directives that enable granular control over the resources a page is allowed to load, including `script-src` from the previous example.

The following list outlines the rest of the resource directives as of level 2. A [level 3 specification](https://www.w3.org/TR/CSP3/) (<https://www.w3.org/TR/CSP3/>) has been drafted, but it's [largely unimplemented](https://www.chromestatus.com/features#csp3) (<https://www.chromestatus.com/features#csp3>) in the major browsers.

base-uri

Restricts the URLs that can appear in a page's `<base>` element.

child-src

Lists the URLs for workers and embedded frame contents. For example, `child-src https://youtube.com` enables embedding videos from YouTube but not from other origins.

connect-src

Limits the origins that you can connect to using XHR, WebSockets, and EventSource.

font-src

Specifies the origins that can serve web fonts. For example, you can allow Google's web fonts using `font-src https://themes.googleusercontent.com`.

form-action

Lists valid endpoints for submission from `<form>` tags.

frame-ancestors

Specifies the sources that can embed the current page. This directive applies to `<frame>`, `<iframe>`, `<embed>`, and `<applet>` tags. It can't be used in `<meta>` tags or for HTML resources.

frame-src

This directive was deprecated in level 2, but is restored in level 3. If it's not present, the browser falls back to `child-src`.

img-src

Defines the origins images can be loaded from.

media-src

Restricts the origins allowed to deliver video and audio.

object-src

Allows control over Flash and other plugins.

plugin-types

Limits the kinds of plugins a page can invoke.

report-uri

Specifies a URL the browser sends reports to when a content security policy is violated. This directive can't be used in `<meta>` tags.

`style-src`

Limits the origins a page can use stylesheets from.

`upgrade-insecure-requests`

Instructs user agents to rewrite URL schemes by changing HTTP to HTTPS. This directive is for websites with large numbers of old URLs that need to be rewritten.

`worker-src`

A CSP Level 3 directive that restricts the URLs that can be loaded as a worker, shared worker, or service worker. As of July 2017, this directive has limited implementations (<https://www.chromestatus.com/features/5922594955984896>).

By default, the browser loads the associated resource from any origin, with no restrictions, unless you set a policy with a specific directive. To override the default, specify a `default-src` directive. This directive defines the defaults for any unspecified directive that ends with `-src`. For example, if you set `default-src` to `https://example.com`, and you don't specify a `font-src` directive, then you can load fonts only from `https://example.com`.

The following directives don't use `default-src` as a fallback. Remember that failing to set them is the same as allowing anything:

- `base-uri`
- `form-action`
- `frame-ancestors`
- `plugin-types`
- `report-uri`
- `sandbox`

Basic CSP syntax

To use CSP directives, list them in the HTTP header with directives separated by colons. Make sure to list all required resources of a specific type in a single directive as follows:

```
script-src https://host1.com https://host2.com
```

The following is an example of multiple directives, in this case for a web app that loads all its resources from a content delivery network at `https://cdn.example.net`, and doesn't use framed content or plugins:

```
Content-Security-Policy: default-src https://cdn.example.net; child-src 'none';
```

Implementation details

Modern browsers support the unprefixed `Content-Security-Policy` header. This is the recommended header. The `X-WebKit-CSP` and `X-Content-Security-Policy` headers you might see in online tutorials are deprecated.

CSP is defined on a page-by-page basis. You'll need to send the HTTP header with every response that you want to protect. This lets you fine-tune the policy for specific pages based on their specific needs. For example, if one set of pages in your site has a +1 button, while others don't, you can allow the button code to load only when necessary.

The source list for each directive is flexible. You can specify sources by scheme (`data:`, `https:`), or ranging in specificity from hostname-only (`example.com`, which matches any origin on that host: any scheme, any port) to a fully qualified URI (`https://example.com:443`, which matches only HTTPS, only `example.com`, and only port 443). Wildcards are accepted, but only as a scheme, a port, or in the leftmost position of the hostname:

`*://*.example.com:*` would match all subdomains of `example.com` (but *not* `example.com` itself), using any scheme, on any port.

The source list also accepts four keywords:

- **'none'** matches nothing.

- **'self'** matches the current origin, but not its subdomains.
- **'unsafe-inline'** allows inline JavaScript and CSS. For more information, see [Avoid inline code](#) (#unsafe-inline).
- **'unsafe-eval'** allows text-to-JavaScript mechanisms like `eval`. For more information, see [Avoid eval\(\)](#) (#unsafe-eval).

These keywords require single-quotes. For example, `script-src 'self'` (with quotes) authorizes the execution of JavaScript from the current host; `script-src self` (no quotes) allows JavaScript from a server named "self" (and *not* from the current host), which probably isn't what you meant.

Sandbox your pages

There's one more directive worth talking about: **sandbox**. It's a bit different from the others we've looked at, as it places restrictions on actions that the page can take rather than on resources that the page can load. If the **sandbox** directive is present, the page is treated as though it was loaded inside of an `<iframe>` with a **sandbox** attribute. This can have a wide range of effects on the page: forcing the page into a unique origin, and preventing form submission, among others. It's a bit beyond the scope of this page, but you can find full details on valid sandboxing attributes in the ["Sandboxing" section of the HTML5 spec](#) (<https://html.spec.whatwg.org/dev/origin.html#sandboxing>).

The meta tag

CSPs preferred delivery mechanism is an HTTP header. It can be useful, however, to set a policy on a page directly in the markup. Do that using a `<meta>` tag with an **http-equiv** attribute:

```
<meta http-equiv="Content-Security-Policy" content="default-src https://cdn.exam
```

This can't be used for **frame-ancestors**, **report-uri**, or **sandbox**.

Avoid inline code

As powerful as the origin-based allowlists used in CSP directives are, they can't solve the biggest threat posed by XSS attacks: inline script injection. If an attacker can inject a script tag that directly contains some malicious payload (such as `<script>sendMyDataToEvilDotCom()` `</script>`), the browser has no way to distinguish it from a legitimate inline script tag. CSP solves this problem by banning inline script entirely.

This ban includes not only scripts embedded directly in `script` tags, but also inline event handlers and `javascript:` URLs. You'll need to move the content of `script` tags into an external file, and replace `javascript:` URLs and `<a ... onclick="[JAVASCRIPT]">` with appropriate `addEventListener()` calls. For example, you might rewrite the following from:

```
<script>
  function doAmazingThings() {
    alert('YOU ARE AMAZING!');
  }
</script>
<button onclick='doAmazingThings();'>Am I amazing?</button>
```

to something more like:

```
<!-- amazing.html -->
<script src='amazing.js'></script>
<button id='amazing'>Am I amazing?</button>
```

```
// amazing.js
function doAmazingThings() {
  alert('YOU ARE AMAZING!');
}
document.addEventListener('DOMContentLoaded', function () {
  document.getElementById('amazing')
```



```
.addEventListener('click', doAmazingThings);  
});
```

The rewritten code is not only compatible with CSP, but also aligned with web design best practices. Inline JavaScript mixes structure and behavior in ways that make code confusing. It's also more complicated to cache and compile. Moving your code into external resources makes your pages perform better.

Moving inline `style` tags and attributes into external stylesheets is also strongly recommended to protect your site against CSS-based [data exfiltration attacks](https://scarybeastsecurity.blogspot.com/2009/12/generic-cross-browser-cross-domain.html) (<https://scarybeastsecurity.blogspot.com/2009/12/generic-cross-browser-cross-domain.html>).

How to temporarily allow inline scripts and styles

Warning: Banning inline scripts and styles is CSP's most important security feature. Only use these methods if a script or style sheet absolutely must be inline.

You can enable inline scripts and styles by adding `'unsafe-inline'` as an allowed source in a `script-src` or `style-src` directive. CSP Level 2 also lets you add specific inline scripts to your allowlist using either a cryptographic nonce (number used once) or hash as follows.

To use a nonce, give your script tag a nonce attribute. Its value must match one in the list of trusted sources. For example:

```
<script nonce="EDNnf03nceI0fn39fn3e9h3sdfa">  
  // Some inline code I can't remove yet, but need to as soon as possible.  
</script>
```

Add the nonce to your `script-src` directive following the `nonce-` keyword:

```
Content-Security-Policy: script-src 'nonce-EDNnf03nceI0fn39fn3e9h3sdfa'
```

Nonces must be regenerated for every page request, and they must be unguessable.

Hashes work in a similar way. Instead of adding code to the script tag, create an SHA hash of the script itself and add it to the `script-src` directive. For example, if your page contained this:

```
<script>alert('Hello, world.');
```

Your policy must contain this:

```
Content-Security-Policy: script-src 'sha256-qznLcsR0x4GACP2dm0UCKCzCG-HiZ1guq6ZZ
```

The `sha*-` prefix specifies the algorithm that generates the hash. The previous example uses `sha256-`, but CSP also supports `sha384-` and `sha512-`. When generating the hash, omit the `<script>` tags. Capitalization and whitespace matter, including leading and trailing whitespace.

Solutions for generating SHA hashes are available in any number of languages. Using Chrome 40 or later, you can open DevTools and then reload your page. The Console tab shows error messages with the correct SHA-256 hash for each of your inline scripts.

Avoid `eval()`

Even when an attacker can't inject script directly, they might be able to trick your application into converting input text into executable JavaScript and executing it on their behalf. `eval()`,

`new Function()`, `setTimeout([string], ...)`, and `setInterval([string], ...)` are all vectors attackers can use to execute malicious code through injected text. CSP's default response to this risk is to completely block all of these vectors.

This has the following effects on the way you build applications:

- You must parse JSON using the built-in `JSON.parse`, instead of relying on `eval`. Safe JSON operations are available in every browser since IE8 (<https://caniuse.com/#feat=json>).
- You must rewrite any `setTimeout` or `setInterval` calls you make using inline functions instead of strings. For example, if your page contains the following:

```
setTimeout("document.querySelector('a').style.display = 'none';", 10);
```

Rewrite it as:

```
setTimeout(function () {  
    document.querySelector('a').style.display = 'none';  
}, 10);  
```
```

- Avoid inline templating at runtime. Many templating libraries use `new Function()` often to speed up template generation at runtime, which allows evaluation of malicious text. Some frameworks support CSP out of the box, falling back to a robust parser in the absence of `eval`. AngularJS's `ng-csp` directive (<https://docs.angularjs.org/api/ng/directive/ngCsp>) is a good example of this. However, we instead recommend using a templating language that offers precompilation, such as Handlebars (<https://handlebarsjs.com/installation/precompilation.html>). Precompiling your templates can make the user experience even faster than the fastest runtime implementation, as well as making your site more secure.

If `eval()` or other text-to-JavaScript functions are essential to your application, you can enable them by adding `'unsafe-eval'` as an allowed source in a `script-src` directive. We strongly discourage this because of the code injection risk it presents.

## Report policy violations

---

To notify the server of bugs that might allow malicious injection, you can tell the browser to POST JSON-formatted violation reports to a location specified in a `report-uri` directive:

```
Content-Security-Policy: default-src 'self'; ...; report-uri /my_amazing_csp_rep
```

These reports look like the following:

```
{
 "csp-report": {
 "document-uri": "http://example.org/page.html",
 "referrer": "http://evil.example.com/",
 "blocked-uri": "http://evil.example.com/evil.js",
 "violated-directive": "script-src 'self' https://apis.google.com",
 "original-policy": "script-src 'self' https://apis.google.com; report-uri ht
 }
}
```

The report contains helpful information for finding the cause of a policy violation, including the page it happened on (`document-uri`), that page's `referrer`, the resource that violated the page's policy (`blocked-uri`), the specific directive it violated (`violated-directive`), and the page's complete policy (`original-policy`).

## Report-Only

If you're just starting out with CSP, we recommend using report-only mode to evaluate the state of your app before you change your policy. To do this, instead of sending a `Content-Security-Policy` header, send a `Content-Security-Policy-Report-Only` header:

```
Content-Security-Policy-Report-Only: default-src 'self'; ...; report-uri /my_ama
```

The policy specified in report-only mode doesn't block restricted resources, but it does send violation reports to the location you specify. You can even send *both* headers, to enforce one policy while monitoring another. This is a great way to test changes to your CSP while enforcing your current policy: turn on reporting for a new policy, monitor the violation reports and fix any bugs, and when you're satisfied with the new policy, start enforcing it.

## Real World Usage

---

The first step towards crafting a policy for your app is to evaluate the resources it loads. When you understand your app's structure, create a policy based on its requirements. The following sections walk through a few common use cases and the decision process for supporting them following the CSP guidelines.

### Social media widgets

- Facebook's [Like button](https://developers.facebook.com/docs/plugins/like-button) (<https://developers.facebook.com/docs/plugins/like-button>) has several implementation options. We recommend using the `<iframe>` version to keep it sandboxed from the rest of your site. It needs a `child-src https://facebook.com` directive to function properly.
- X's [Tweet button](https://publish.twitter.com/) (<https://publish.twitter.com/>) relies on access to a script. Move the script it provides into an external JavaScript file, and use the directive `script-src https://platform.twitter.com; child-src https://platform.twitter.com`.
- Other platforms have similar requirements, and can be addressed similarly. To test these resources, we recommend setting a `default-src` of `'none'` and watching your console

to determine which resources you'll need to enable.

To use multiple widgets, combine the directives as follows:

```
script-src https://apis.google.com https://platform.twitter.com; child-src https
```

## Lockdown

For some websites, you'll want to make sure that only local resources can be loaded. The following example develops a CSP for a banking site, starting with a default policy that blocks everything (`default-src 'none'`).

The site loads all images, style, and script from a CDN at `https://cdn.mybank.net`, and connects to `https://api.mybank.com/` using XHR to retrieve data. It uses frames, but only for pages local to the site (no third-party origins). There's no Flash on the site, no fonts, no extras. The most restrictive CSP header it can send is this:

```
Content-Security-Policy: default-src 'none'; script-src https://cdn.mybank.net;
```

## SSL only

The following is an example CSP for a forum administrator who wants to ensure that all resources on their forum are only loaded using secure channels, but is inexperienced at coding and doesn't have the resources to rewrite third-party forum software full of inline scripts and styles:

```
Content-Security-Policy: default-src https;; script-src https: 'unsafe-inline';
```

Although `https:` is specified in `default-src`, the script and style directives don't automatically inherit that source. Each directive overwrites the default for that specific type of resource.

## CSP standard development

---

Content Security Policy Level 2 is a W3C recommended standard (<https://www.w3.org/TR/CSP2/>). W3C's Web Application Security Working Group is developing the specification's next iteration, Content Security Policy Level 3 (<https://www.w3.org/TR/CSP3/>).

To follow the discussion around these upcoming features, refer to the public-webappsec@ mailing list archives (<http://lists.w3.org/Archives/Public/public-webappsec/>).

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the Apache 2.0 License (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the Google Developers Site Policies (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2012-06-15 UTC.